

# CS-338 Intro to the Theory of Computation

- Lecture #2

## ➤ **Nondeterministic Finite Automata (NFAs)**

- Prof Pietro S. Oliveto

Department of Computer Science and Engineering

Southern University of Science and Technology (SUSTech)

- `oliveto@ustech.edu.cn`  
<https://faculty.ustech.edu.cn/oliveto>
- Office: Room 113

# Lecture 2: Overview

## **Last time:** (Sipser §1.1)

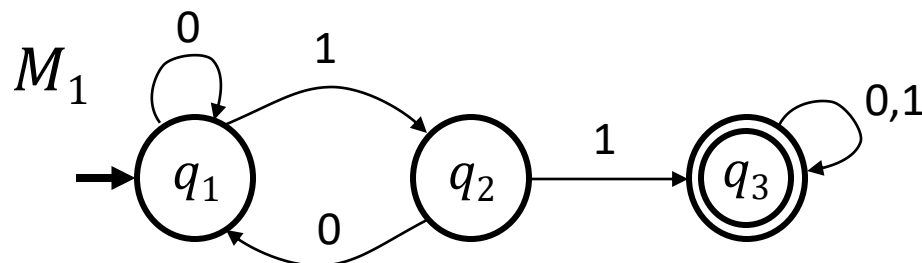
- Finite automata, regular languages
- Regular operations  $\cup, \circ, *$
- Regular expressions
- Closure under  $\cup$

## **Today:** (Sipser §1.2 – §1.3)

- Nondeterminism
- Closure under  $\circ$  and  $*$

**Goal:** Show finite automata equivalent to regular expressions

# Finite Automata (Recap)



States:  $q_1 q_2 q_3$

Transitions:  $\xrightarrow{1}$

Start state:  $\rightarrow \bigcirc$

Accept states:  $\bigcirc\bigcirc$

**Input:** finite string

**Output:** Accept or Reject

**Computation process:** Begin at start state, read input symbols, follow corresponding transitions, Accept if end with accept state, Reject if not.

**Examples:** 01101  $\rightarrow$  Accept  
00101  $\rightarrow$  Reject

$M_1$  accepts exactly those strings in  $A$  where  
 $A = \{w \mid w \text{ contains substring } 11\}.$

Say that  $A$  is the language of  $M_1$  and that  $M_1$  recognizes  $A$  and that  $A = L(M_1).$

A language is **regular** if some finite automaton recognises it

# Operations on languages (recap)

**Regular operations.** Let  $A, B$  be languages:

- Union:  $A \cup B = \{w \mid w \in A \text{ or } w \in B\}$
- Concatenation:  $A \circ B = \{xy \mid x \in A \text{ and } y \in B\} = AB$
- Star:  $A^* = \{x_1 \dots x_k \mid \text{each } x_i \in A \text{ for } k \geq 0\}$   
Note:  $\varepsilon \in A^*$  always

**Example.** Let  $A = \{\text{good, bad}\}$  and  $B = \{\text{boy, girl}\}$ .

- $A \cup B = \{\text{good, bad, boy, girl}\}$
- $A \circ B = AB = \{\text{goodboy, goodgirl, badboy, badgirl}\}$
- $A^* = \{\varepsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, ...}\}$

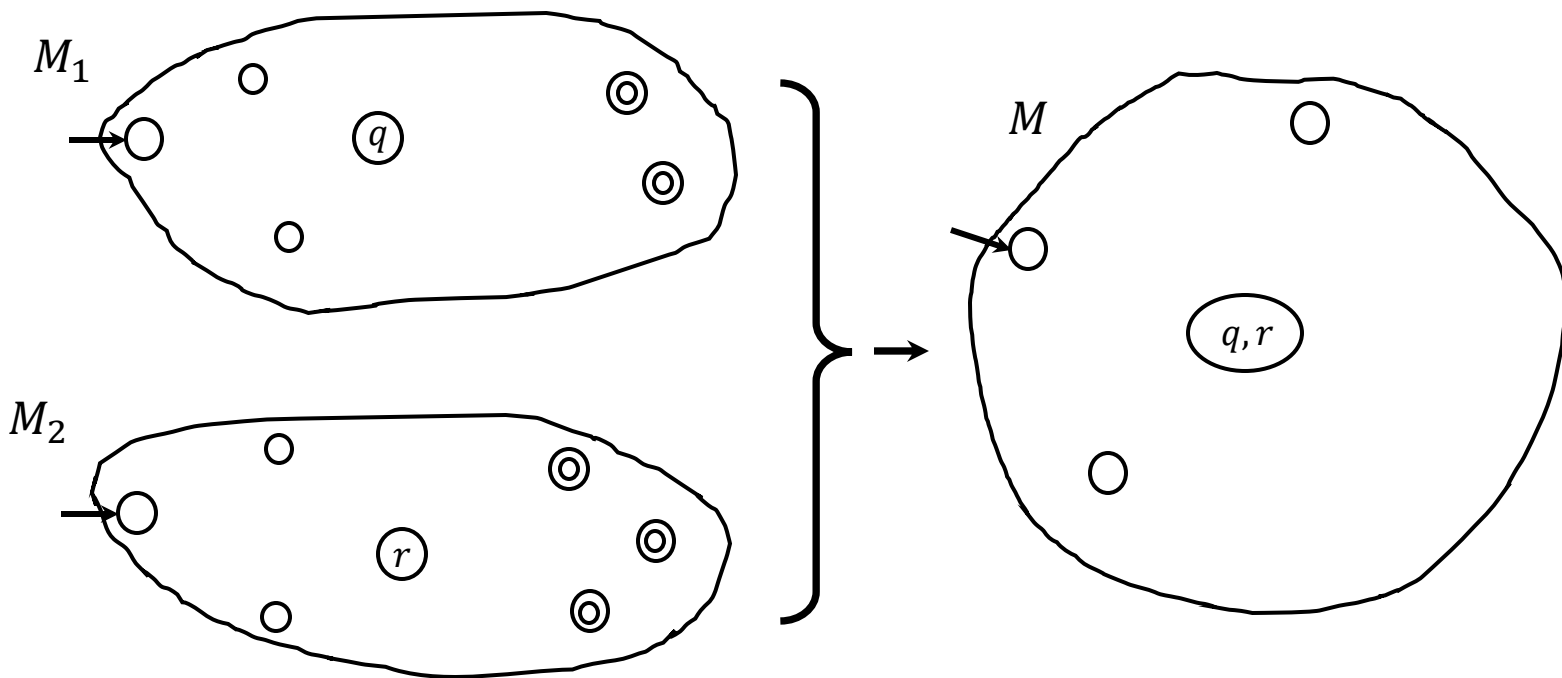
# Closure Properties: Union (Recap)

**Theorem:** If  $A_1, A_2$  are regular languages, so is  $A_1 \cup A_2$  (closure under  $\cup$ )

**How to prove:** Let  $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  recognize  $A_1$   
Let  $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$  recognize  $A_2$

Construct  $M = (Q, \Sigma, \delta, q_0, F)$  that recognizes  $A_1 \cup A_2$

$M$  should accept input  $w$  if either  $M_1$  or  $M_2$  accept  $w$ .



**Components of  $M$ :**

$$Q = Q_1 \times Q_2 \\ = \{(q_1, q_2) \mid q_1 \in Q_1 \text{ and } q_2 \in Q_2\}$$

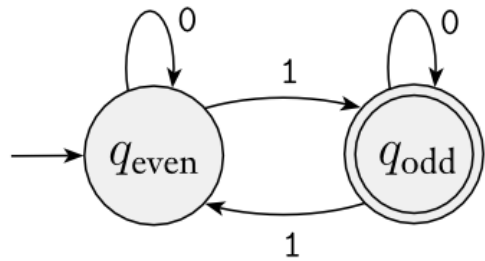
$$q_0 = (q_1, q_2)$$

$$\Sigma$$

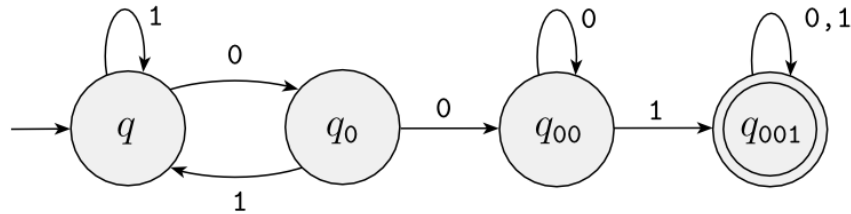
$$\delta((q, r), a) = (\delta_1(q, a), \delta_2(r, a))$$

$$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

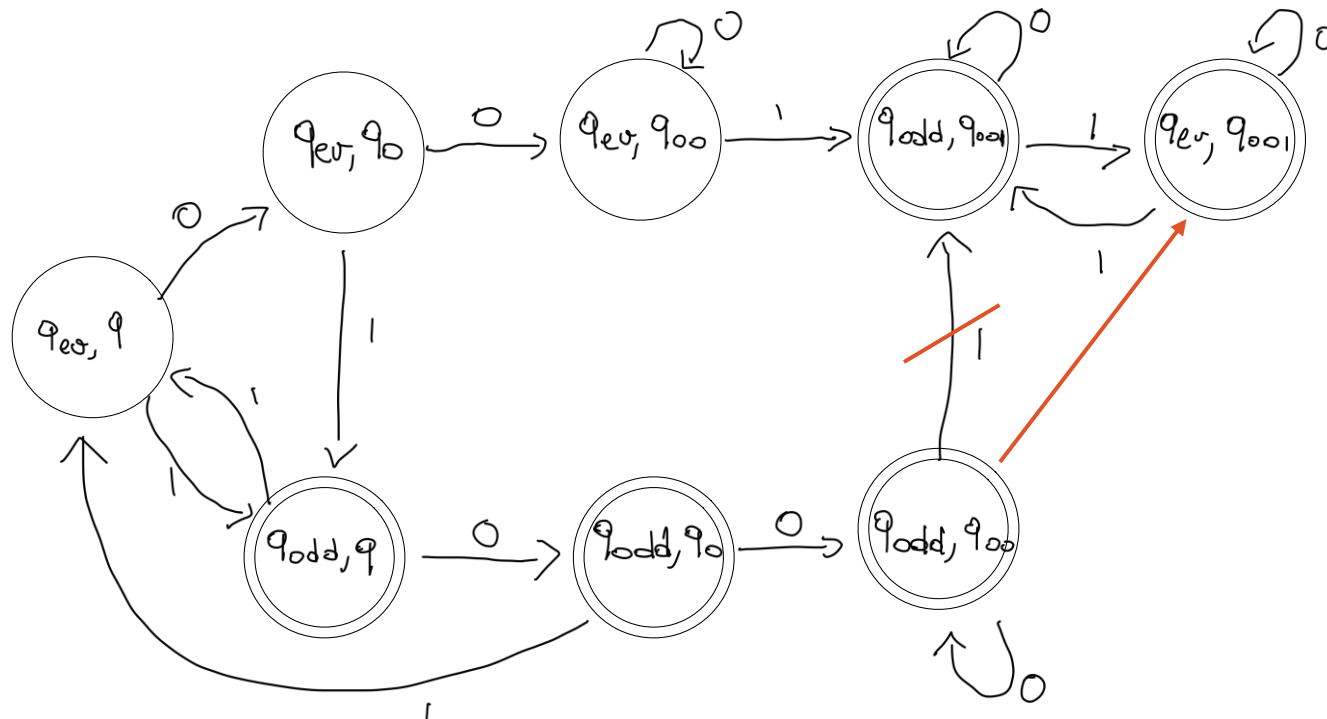
# Closure Properties: Union Example



Language  $D = \{w \mid w \text{ has an odd number of 1s}\}$



Language  $E = \{w \mid w \text{ contains 001 as a substring}\}$



Language  $D \cup E = \{w \mid w \text{ has an odd number of 1s or } w \text{ contains 001 as a substring}\}$

Components of  $M$ :

$$Q = Q_1 \times Q_2, \quad |Q| = 8$$

$$q_0 = (q_{\text{ev}}, q)$$

$\Sigma$

$$\delta((q, r), a) = (\delta_1(q, a), \delta_2(r, a))$$

$$F = (F_1 \times Q_2) \cup (Q_1 \times F_2), \quad |F| = 5$$

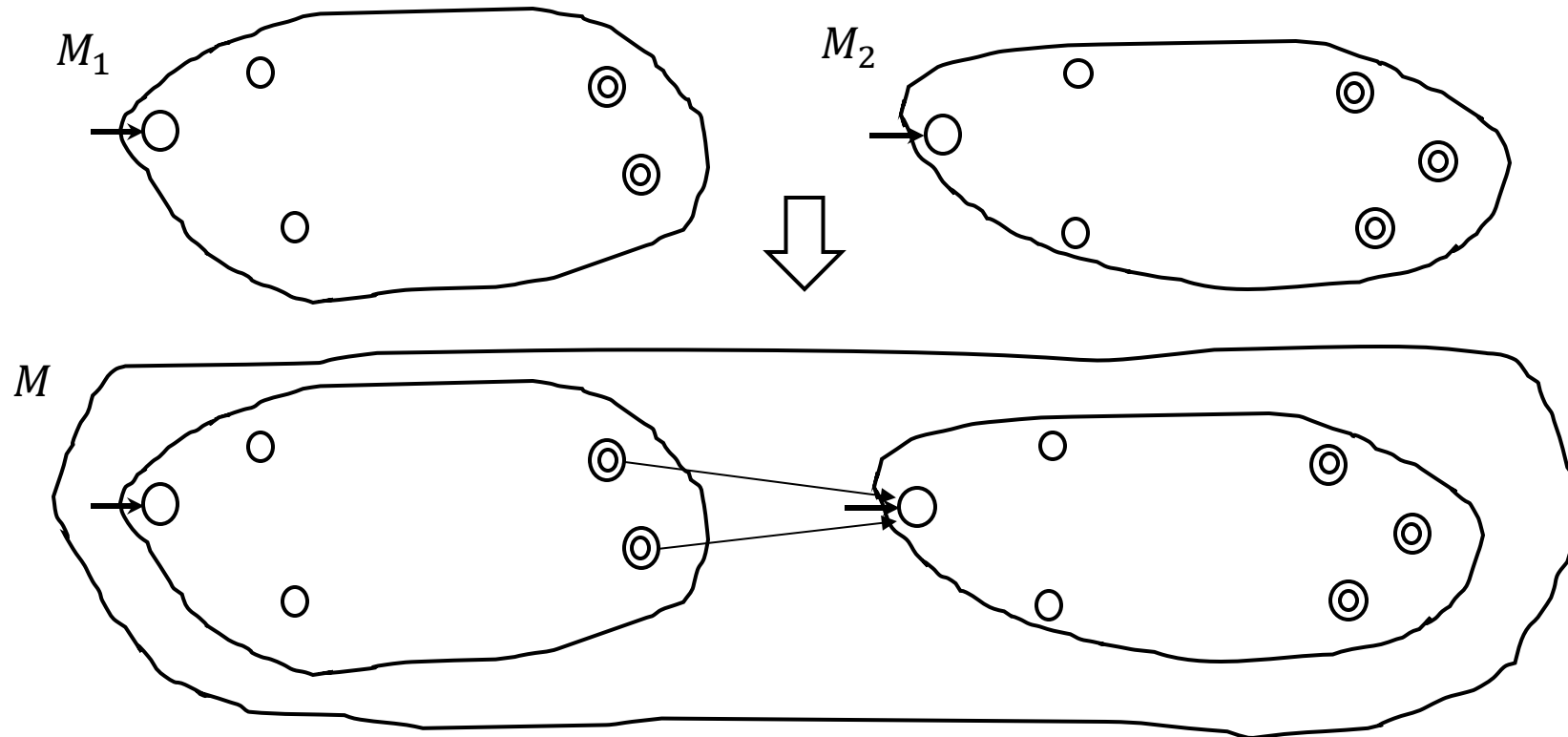
# Closure Properties: Concatenation

**Theorem:** If  $A_1, A_2$  are regular languages, so is  $A_1A_2$  (closure under  $\circ$ )

**Recall proof attempt:** Let  $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  recognize  $A_1$

$$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$$

Construct  $M = (Q, \Sigma, \delta, q_0, F)$



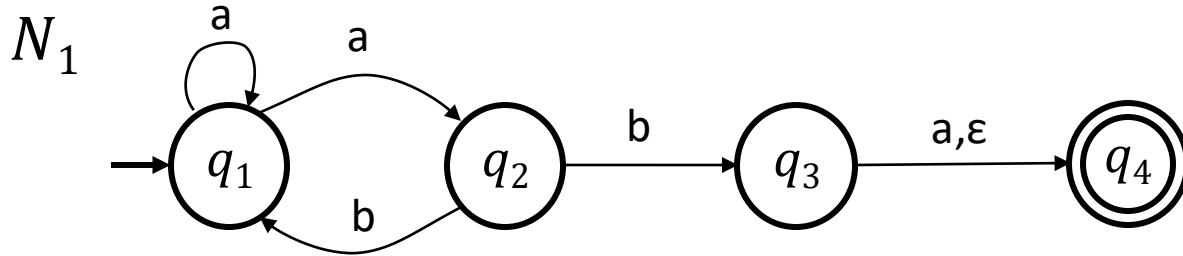
$M$  should accept input  $w$   
if  $w = xy$  where  
 $M_1$  accepts  $x$  and  $M_2$  accepts  $y$ .

$w$   $\xrightarrow{x}$   $\xrightarrow{y}$

Doesn't work: Where to split  $w$ ?

Hold off. Need new concept.

# Nondeterministic Finite Automata



## New features of nondeterminism:

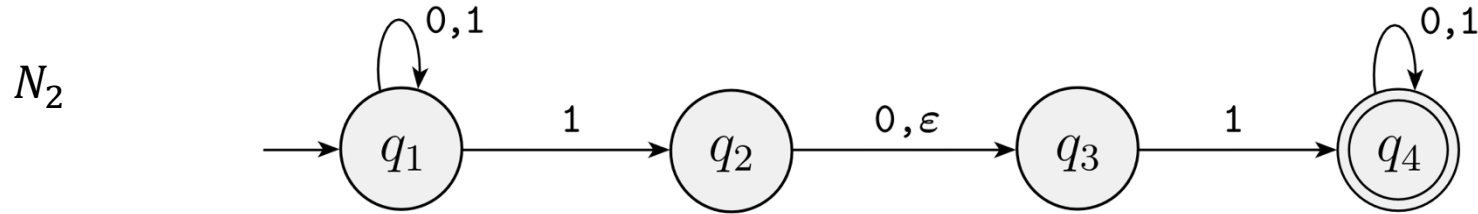
- multiple paths possible (0, 1 or many at each step)  
(DFAs have exactly one transition for each alphabet symbol: NFAs may have more than one or zero);
- $\epsilon$ -transition is a “free” move without reading input;
- All paths are explored in parallel: if multiple choices are available,  $N_1$  creates multiple copies of itself one per choice
- If the next symbol does not appear as a transition from the current state  $\Rightarrow$  that computation dies;
- Accept input if some path leads to  $\odot$  accept

## Example inputs:

- ab accept
- aa reject
- aba accept
- abb reject

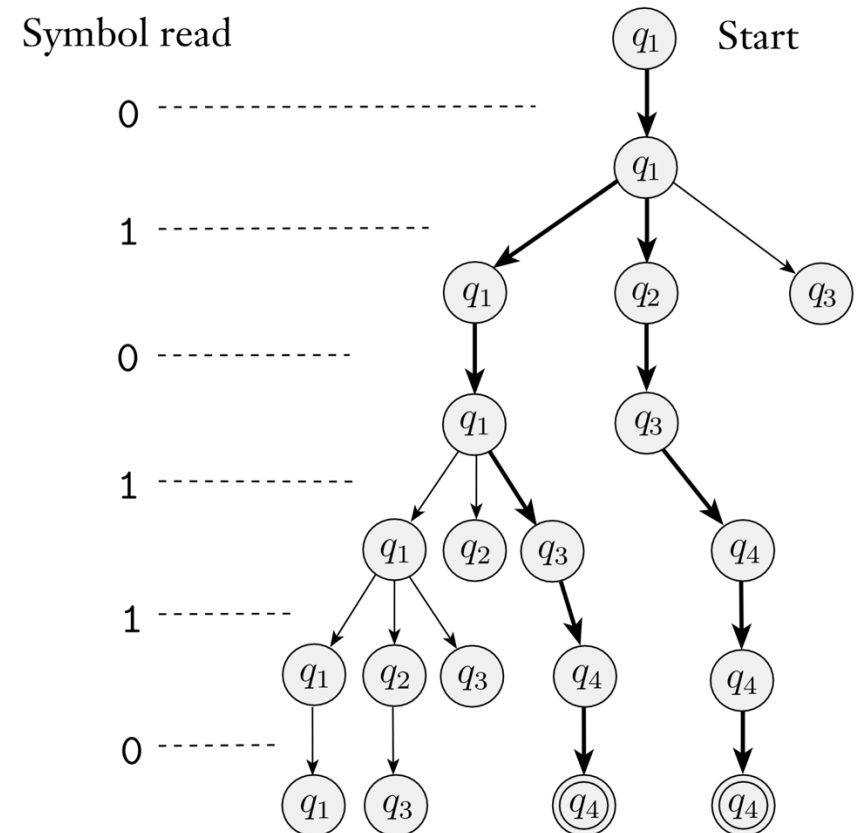
Nondeterminism doesn't correspond  
to a physical machine we can build.  
However, it is useful mathematically.

# NFA: example



- Nondeterminism can be viewed as a kind of parallel computation where multiple “processes” or “threads” can be running concurrently
- When the NFA splits to follow several choices, that corresponds to a process “forking” into several children, each proceeding separately
- If at least one of these processes accepts, then the whole computation accepts

Input: 010110

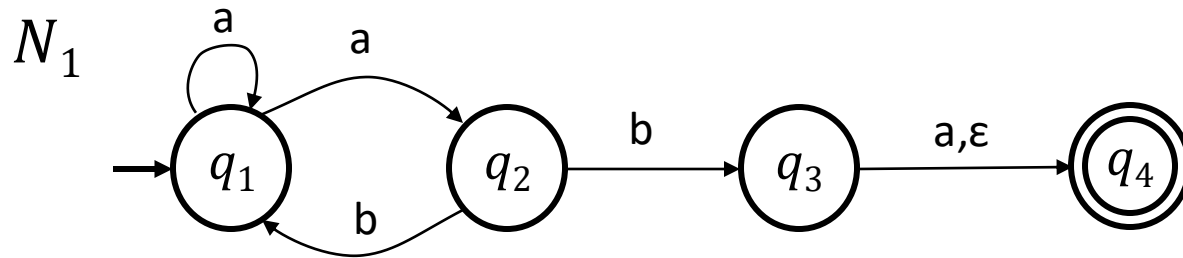


What does  $N_2$  do on input 010?

## What language does $N_2$ recognise?

$$L(N_2) = \{w \mid w \text{ contains either } 101 \text{ or } 11 \text{ as a substring}\}$$

# NFA – Formal Definition



**Defn:** A nondeterministic finite automaton (NFA)

$N$  is a 5-tuple  $(Q, \Sigma, \delta, q_0, F)$

states  
alphabet  
transition function  
start state  
accept states

- all same as before except  $\delta$
- $\delta: Q \times \underbrace{\Sigma \cup \{\epsilon\}}_{\text{power set}} \rightarrow \mathcal{P}(Q) = \{R | R \subseteq Q\}$
- In the  $N_1$  example:  $\delta(q_1, a) = \{q_1, q_2\}$   
 $\delta(q_1, b) = \emptyset$

(A power set  $\mathcal{P}(Q)$  is the collection of all possible subsets of  $Q$ )

**Ways to think about nondeterminism:**

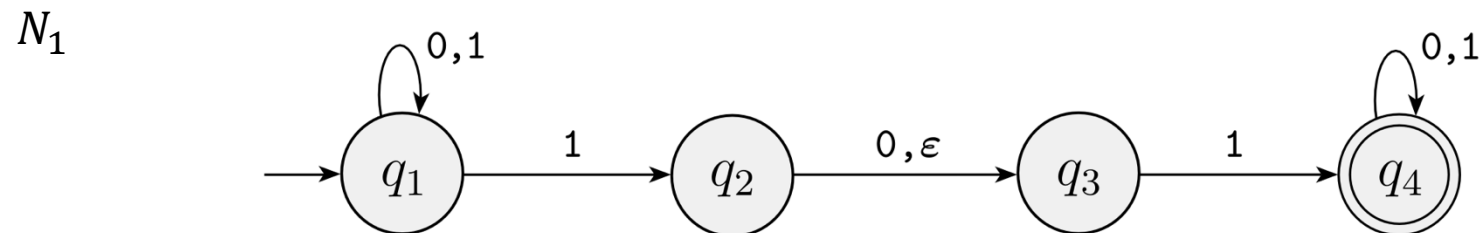
**Computational:** Fork new parallel thread and accept if any thread leads to an accept state.

**Mathematical:** Tree with branches.

Accept if any branch leads to an accept state.

**Magical:** Guess at each nondeterministic step which way to go. Machine always makes the right guess that leads to accepting, if possible.

# NFA – Formal Definition: Example



The formal description of  $N_1$  is  $(Q, \Sigma, \delta, q_1, F)$ , where

1.  $Q = \{q_1, q_2, q_3, q_4\}$ ,
2.  $\Sigma = \{0,1\}$ ,
3.  $\delta$  is given as

|       | 0           | 1              | $\epsilon$  |
|-------|-------------|----------------|-------------|
| $q_1$ | $\{q_1\}$   | $\{q_1, q_2\}$ | $\emptyset$ |
| $q_2$ | $\{q_3\}$   | $\emptyset$    | $\{q_3\}$   |
| $q_3$ | $\emptyset$ | $\{q_4\}$      | $\emptyset$ |
| $q_4$ | $\{q_4\}$   | $\{q_4\}$      | $\emptyset$ |

4.  $q_1$  is the start state, and
5.  $F = \{q_4\}$ .

## Computation

Let  $N = (Q, \Sigma, \delta, q_0, F)$  be an NFA  
and let  $w = w_1w_2 \dots w_n$  with each  $w_i \in \Sigma$

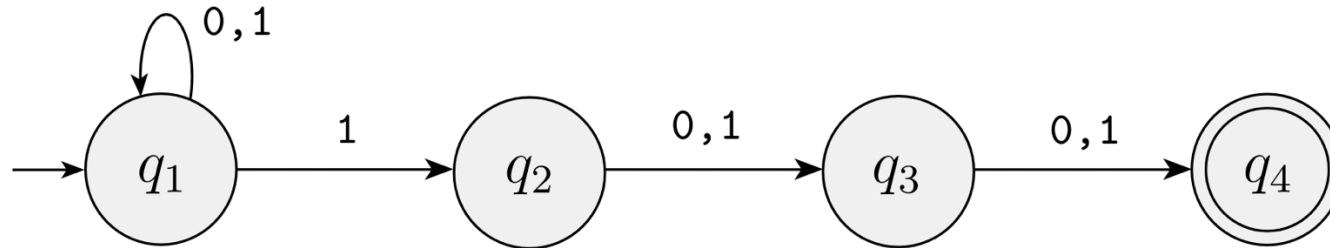
**Defn:**  $M$  accepts string  $w = w_1w_2 \dots w_n$  each  $w_i \in \Sigma$  if we can write  $w$  as  $w = y_1y_2 \dots y_m$  where each  $y_i \in \Sigma_\epsilon$ ,  $m \geq n$  and

there is a sequence of states  $r_0, r_1, r_2, \dots, r_m \in Q$  where:

1.  $r_0 = q_0$
2.  $r_i \in \delta(r_{i-1}, y_i)$  for  $1 \leq i \leq m$
3.  $r_m \in F$

# Designing NFAs: Example

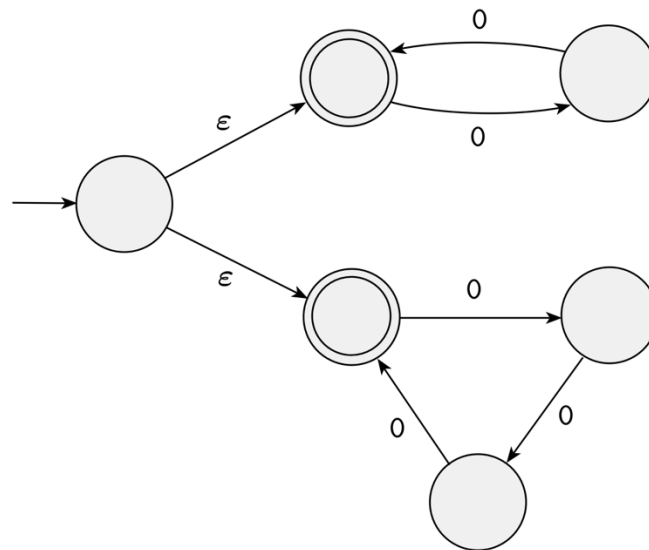
- Design an NFA  $N_3$  that recognises the language over  $\{0,1\}$  containing a 1 in the third position from the end



- It stays in the start state until it "guesses" that it is three places from the end ("**magical**")

# Designing NFAs: Example (2)

- Design an NFA  $N_3$  that recognises the language over the unary alphabet  $\Sigma = \{0\}$  containing strings having either a multiple of two 0s or a multiple of three 0s



- It accepts strings of the form  $0^k$  where  $k$  is either a multiple of 2 or 3
- The machine designed this way is the easiest one to understand what language it recognises

# Which are more powerful NFAs or DFAs?

- Nondeterminism is a generalisation of determinism, so every DFA is automatically also a NFA
- Obviously there are NFAs that are not DFAs
- Can NFAs recognise languages that DFAs cannot recognise?
- DFAs and NFAs recognise the same class of languages
- This is **surprising** because NFAs appear to have more power than DFAs
- This is **useful** because describing a language using NFAs is much easier than using DFAs
- We say that two machines are **equivalent** if they recognise the same language
- **Next:** we prove that every NFA has an equivalent DFA

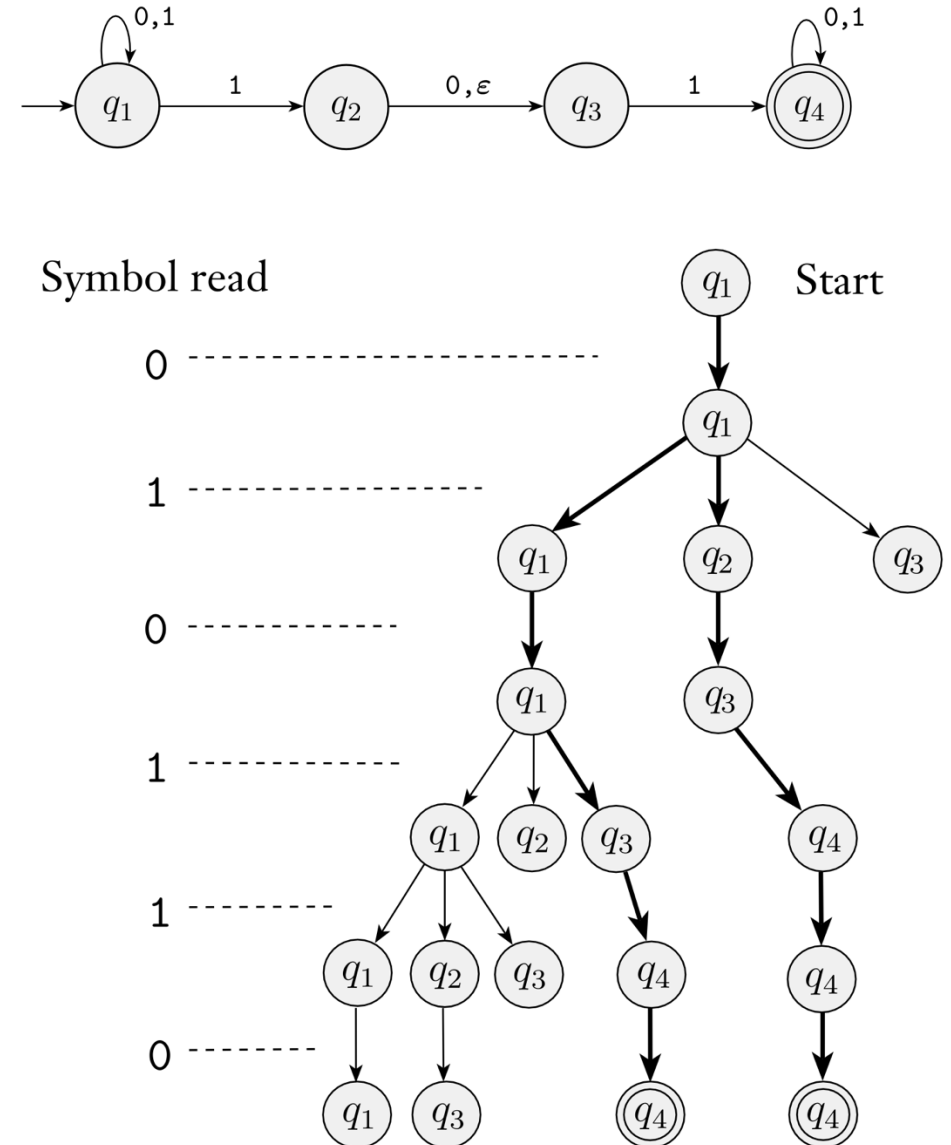
# Converting NFAs to DFAs

**Theorem:** If an NFA recognizes  $A$  then  $A$  is regular

**Proof Idea:** Let NFA  $M = (Q, \Sigma, \delta, q_0, F)$  recognize  $A$

Construct DFA  $M' = (Q', \Sigma, \delta', q'_0, F')$  that recognizes  $A$

- We convert the NFA into an equivalent DFA that **simulates** the NFA
- When you simulate the NFA what do you keep track of at each character of the input?
- You kept track of the various branches of the computation by remembering all the states the machine could be in
- => The DFA needs to keep track of the **set of states** the different branches of computation may be in
- Each possible subset of states is one of the possibilities the DFA must remember
- If  $k$  is the number of states in  $M$ . How many subsets of states are there?  **$2^k$**  (can prove by induction)



# Converting NFAs to DFAs (2)

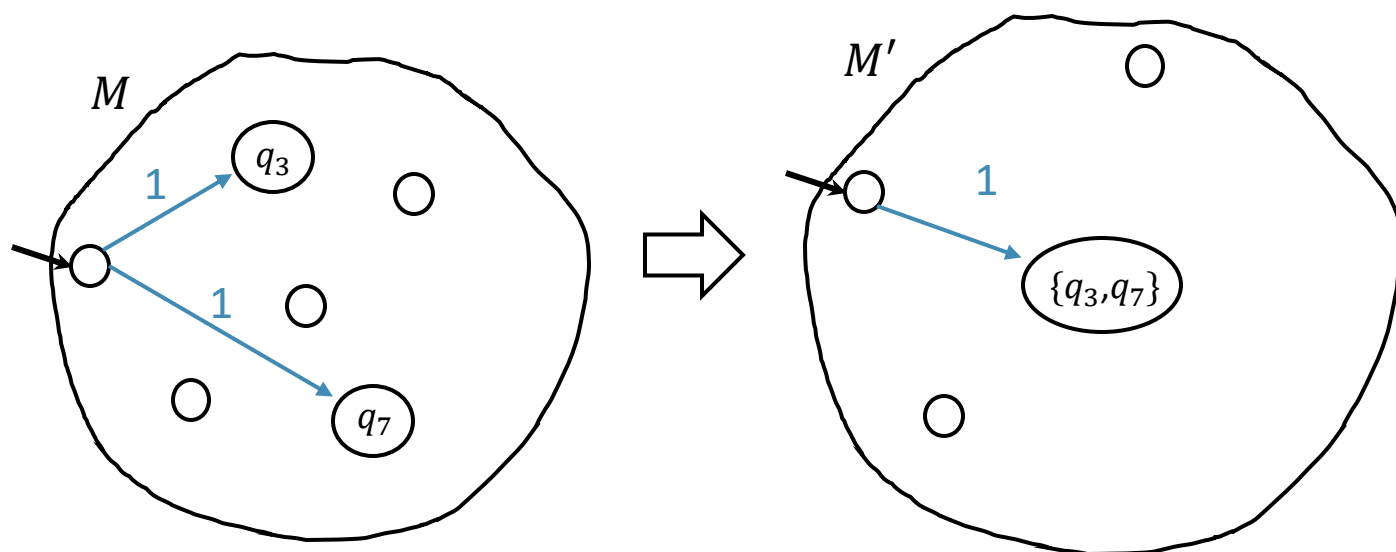
**Theorem:** If an NFA recognizes  $A$  then  $A$  is regular

**Proof Idea:** Let NFA  $M = (Q, \Sigma, \delta, q_0, F)$  recognize  $A$

Construct DFA  $M' = (Q', \Sigma, \delta', q'_0, F')$  that recognizes  $A$

(Ignore the  $\epsilon$ -transitions, can easily modify to handle them)

**IDEA:** DFA  $M'$  keeps track of the subset of possible states in NFA  $M$ .



**Construction of  $M'$ :**

$$Q' = \mathcal{P}(Q)$$

$$\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ for some } r \in R\}$$
$$\overbrace{R \in Q'}^{\text{}}. \delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

$$q'_0 = \{q_0\}$$

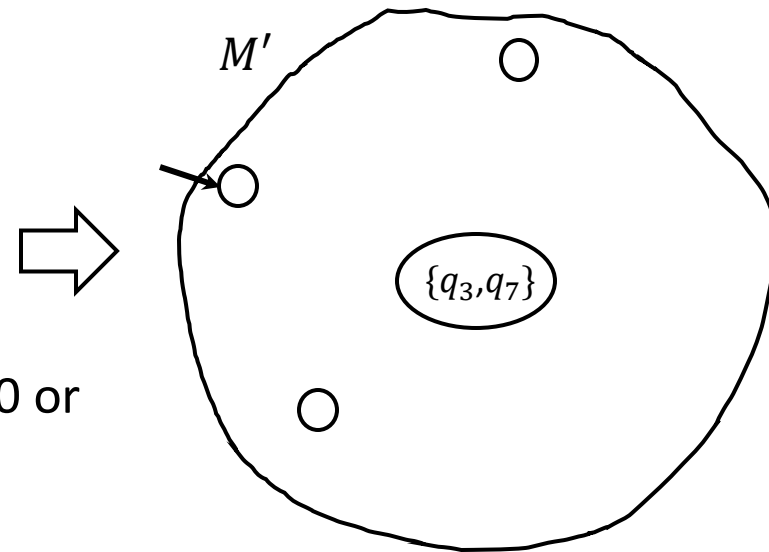
$$F' = \{R \in Q' \mid R \text{ intersects } F\}$$

# Converting NFAs to DFAs (3): $\varepsilon$ transitions

**Theorem:** If an NFA recognizes  $A$  then  $A$  is regular

**Proof Idea:** Let NFA  $M = (Q, \Sigma, \delta, q_0, F)$  recognize  $A$   
Construct DFA  $M' = (Q', \Sigma, \delta', q'_0, F')$  that recognizes  $A$

- For any  $R \in Q'$ , let  $E(R) = \{q \mid q \text{ can be reached from } R \text{ by travelling on 0 or more } \varepsilon \text{ arrows, including members of } R \text{ themselves}\}$



Construction of  $M'$  (no  $\varepsilon$ ):

$$Q' = \mathcal{P}(Q)$$

$$\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ for some } r \in R\}$$

$$\bigwedge_{R \in Q'} \delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

$$q'_0 = \{q_0\}$$

$$F' = \{R \in Q' \mid R \text{ intersects } F\}$$

Construction of  $M'$  (with  $\varepsilon$ ):

$$Q' = \mathcal{P}(Q)$$

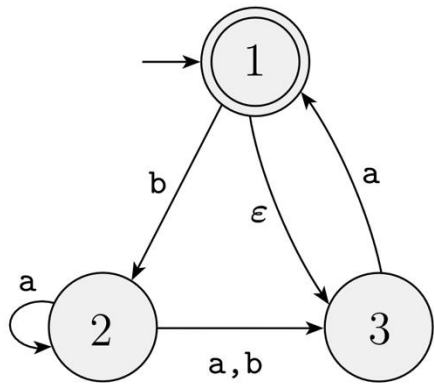
$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ for some } r \in R\}$$

(after every step)

$$q'_0 = \{E(q_0)\}$$

$$F' = \{R \in Q' \mid R \text{ intersects } F\}$$

# Converting NFAs to DFAs: Example



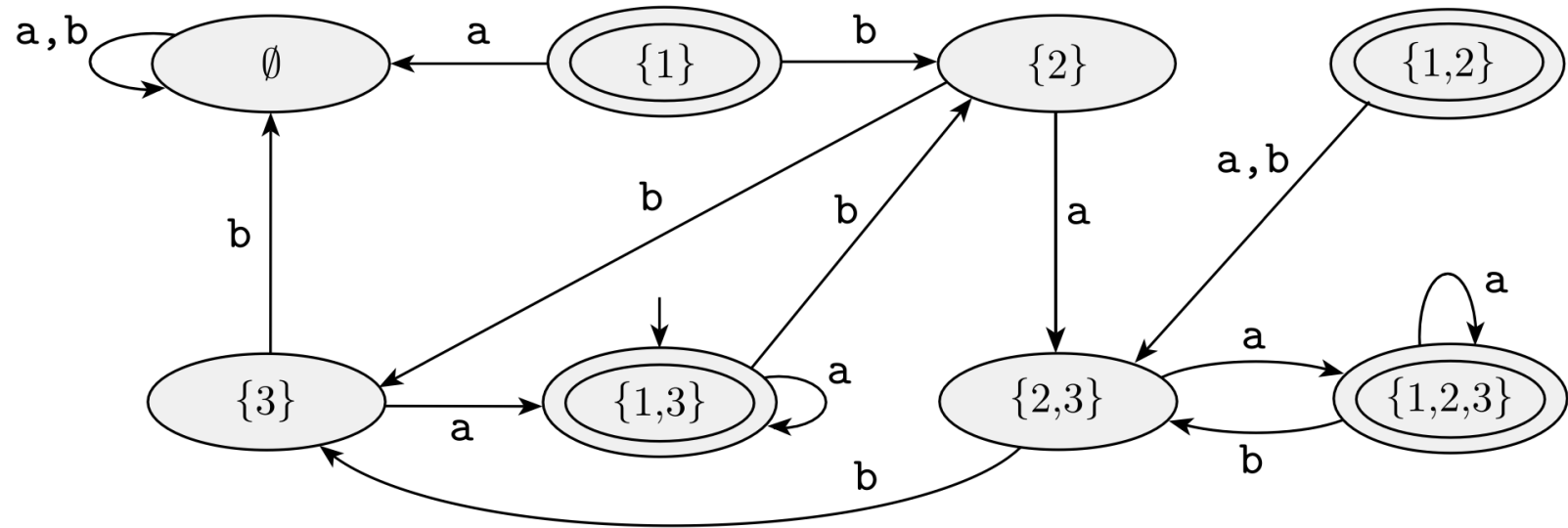
$$N_4 = (Q, \{a, b\}, \delta, 1, \{1\})$$

Construct equivalent DFA D.

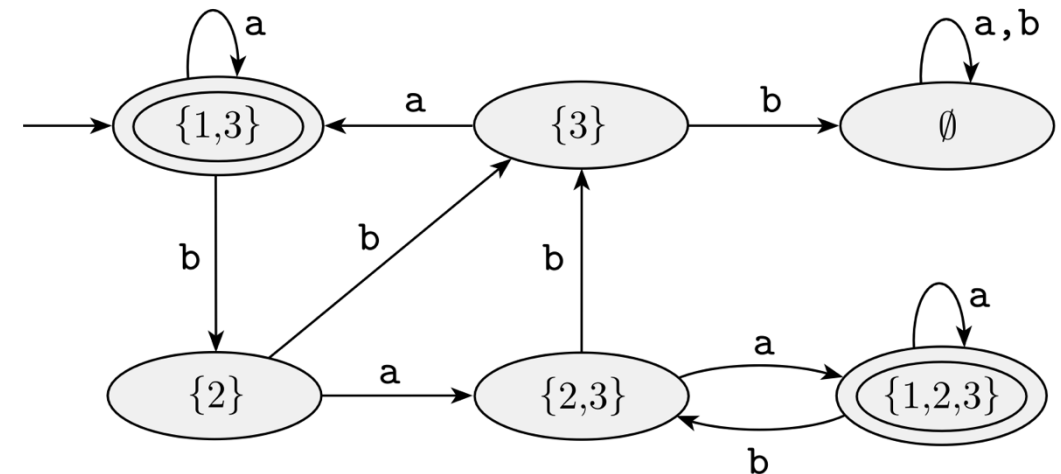
$$Q' = \mathcal{P}(Q) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$$

$$q'_0 = \{1,3\} \text{ (the set of states reachable from 1)}$$

$$F' = \{\{1\}, \{1,2\}, \{1,3\}, \{1,2,3\}\}$$



And, removing unnecessary states:



# Consequence of equivalence

**Corollary:** A language is regular if and only if some nondeterministic finite automaton recognizes it

( $\Rightarrow$ ) if a language is regular then an NFA can recognise it

( $\Leftarrow$ ) If an NFA can recognise a language, then it is regular

**Proof:**

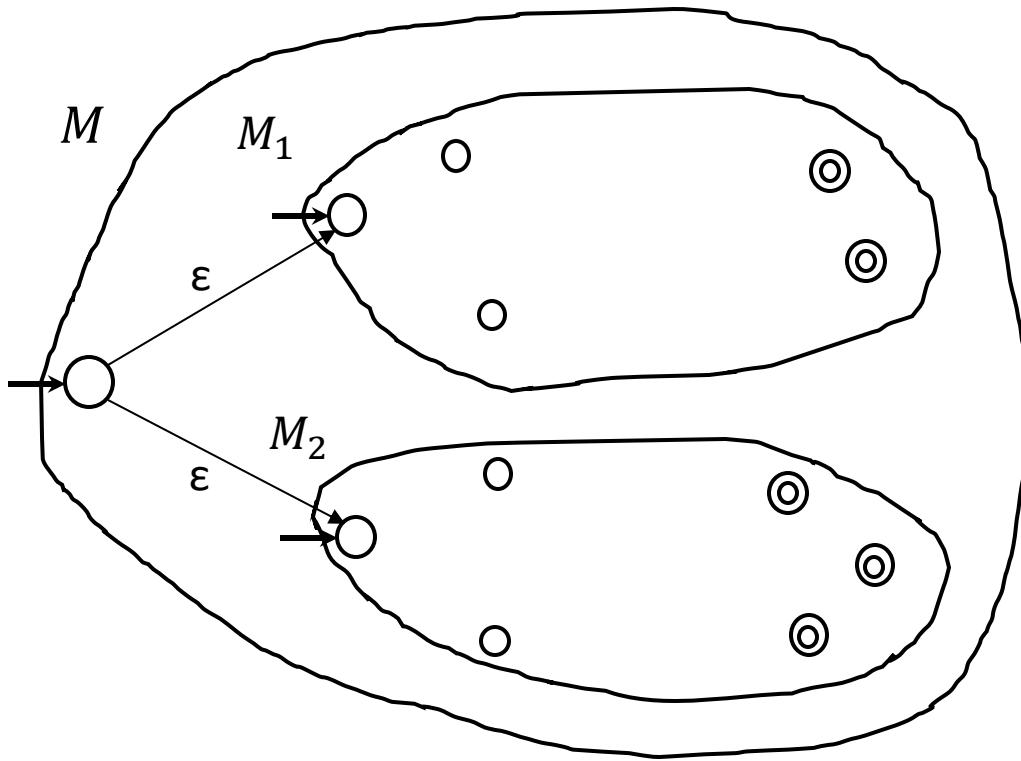
- ( $\Rightarrow$ ) A language is regular, then an NFA recognises it
  - Trivial: an NFA is a DFA and true by definition
- ( $\Leftarrow$ ) If an NFA can recognise a language, then it is regular
  - The Theorem states that there exists a DFA that is equivalent to the NFA. Thus the language is regular by definition.

# Return to Closure Properties

Recall Theorem:

(The class of regular languages is closed under union)

New Proof (sketch):



# Closure under Union

Let  $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  recognize  $A_1$ , and  
 $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$  recognize  $A_2$ .

Construct  $N = (Q, \Sigma, \delta, q_0, F)$  to recognize  $A_1 \cup A_2$ .

1.  $Q = \{q_0\} \cup Q_1 \cup Q_2$ .

The states of  $N$  are all the states of  $N_1$  and  $N_2$ , with the addition of a new start state  $q_0$ .

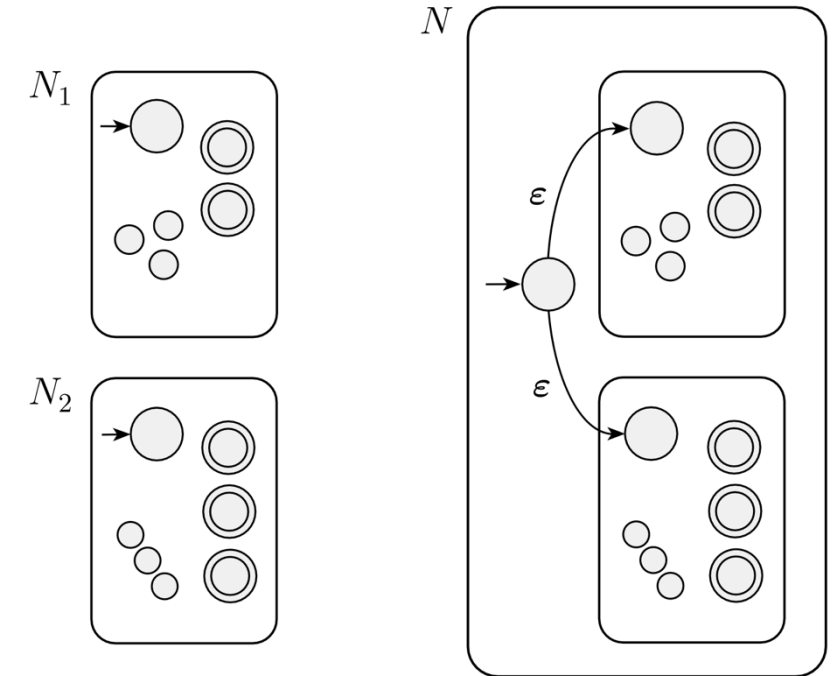
2. The state  $q_0$  is the start state of  $N$ .

3. The set of accept states  $F = F_1 \cup F_2$ .

The accept states of  $N$  are all the accept states of  $N_1$  and  $N_2$ . That way,  $N$  accepts if either  $N_1$  accepts or  $N_2$  accepts.

4. Define  $\delta$  so that for any  $q \in Q$  and any  $a \in \Sigma_\epsilon$ ,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ and } a = \epsilon \\ \emptyset & q = q_0 \text{ and } a \neq \epsilon. \end{cases}$$

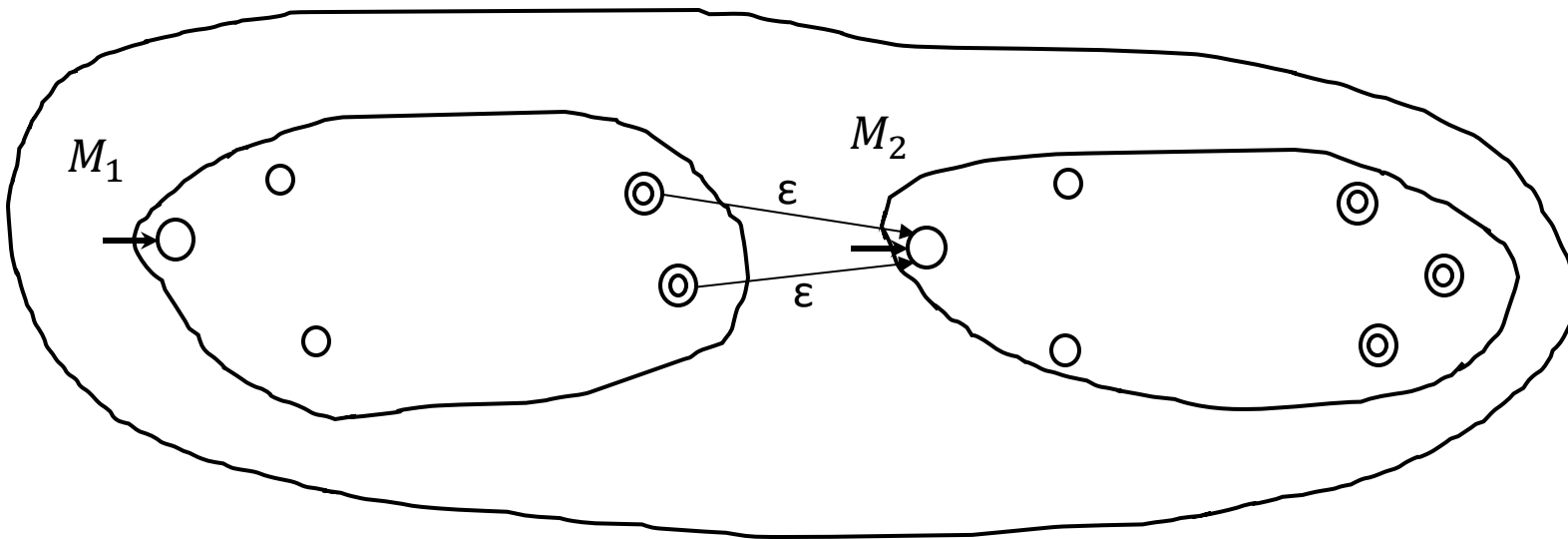


The new machine non-deterministically guesses which of the two machines accepts the input!

# Closure under $\circ$ (concatenation)

**Theorem:** If  $A_1, A_2$  are regular languages, so is  $A_1A_2$

**Proof sketch:** Given DFAs  $M_1$  and  $M_2$  recognizing  $A_1$  and  $A_2$   
Construct NFA  $M$  recognizing  $A_1A_2$



$M$  should accept input  $w$   
if  $w = xy$  where  
 $M_1$  accepts  $x$  and  $M_2$  accepts  $y$ .

$$w = \underbrace{\hspace{1.5cm}}_x \mid \underbrace{\hspace{1.5cm}}_y$$

Nondeterministic  $M'$  has the option  
to jump to  $M_2$  when  $M_1$  accepts.

# Closure under Concatenation

Let  $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  recognize  $A_1$ , and  
 $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$  recognize  $A_2$ .

Construct  $N = (Q, \Sigma, \delta, q_1, F_2)$  to recognize  $A_1 \circ A_2$ .

1.  $Q = Q_1 \cup Q_2$ .

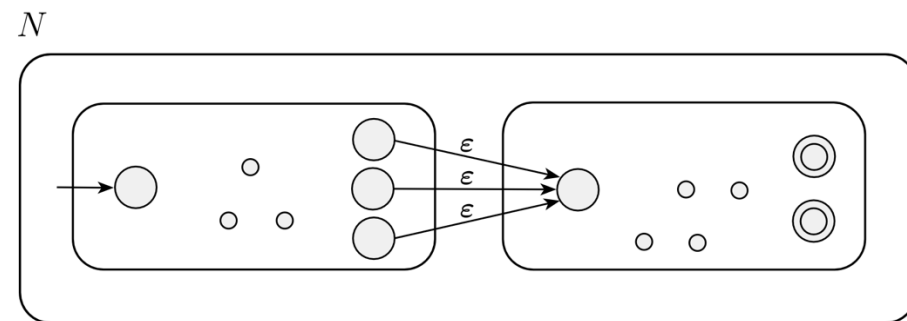
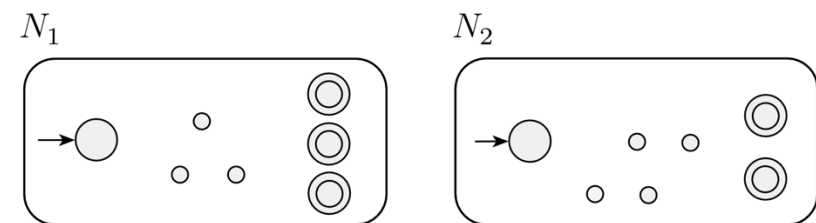
The states of  $N$  are all the states of  $N_1$  and  $N_2$ .

2. The state  $q_1$  is the same as the start state of  $N_1$ .

3. The accept states  $F_2$  are the same as the accept states of  $N_2$ .

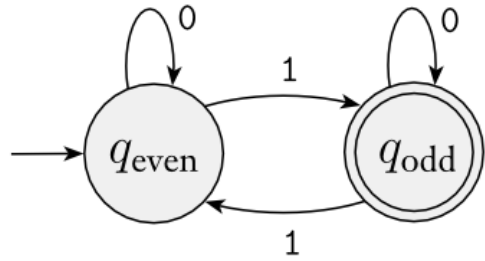
4. Define  $\delta$  so that for any  $q \in Q$  and any  $a \in \Sigma_\epsilon$ ,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ and } a = \epsilon \\ \delta_2(q, a) & q \in Q_2. \end{cases}$$

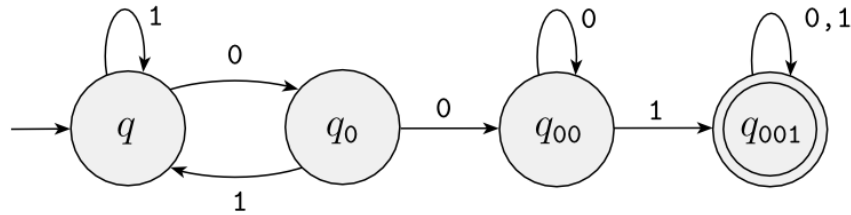


We can think of new machine non-deterministically guessing where to split the input!

# Closure Properties: Concatenation Example



Language  $D = \{w \mid w \text{ has an odd number of 1s}\}$



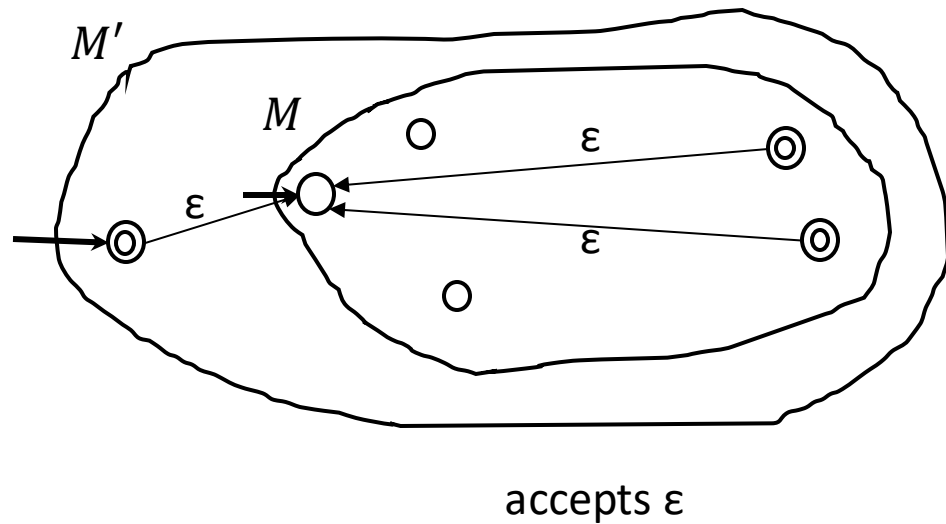
Language  $E = \{w \mid w \text{ contains } 001 \text{ as a substring}\}$

Language  $D \circ E = \{w \mid w = xy \text{ where } x \text{ has an odd number of 1s and } y \text{ contains } 001 \text{ as a substring}\}$

# Closure under $*$ (star)

**Theorem:** If  $A$  is a regular language, so is  $A^*$

**Proof sketch:** Given DFA  $M$  recognizing  $A$   
Construct NFA  $M'$  recognizing  $A^*$



$M'$  should accept input  $w$   
if  $w = x_1x_2 \dots x_k$   
where  $k \geq 0$  and  $M$  accepts each  $x_i$

$$w = \begin{array}{c} \text{---} | \text{---} | \text{---} | \text{---} \\ x_1 \quad x_2 \quad x_3 \quad x_4 \end{array}$$

# Closure under Star

**PROOF** Let  $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  recognize  $A_1$ .

Construct  $N = (Q, \Sigma, \delta, q_0, F)$  to recognize  $A_1^*$ .

1.  $Q = \{q_0\} \cup Q_1$ .

The states of  $N$  are the states of  $N_1$  plus a new start state.

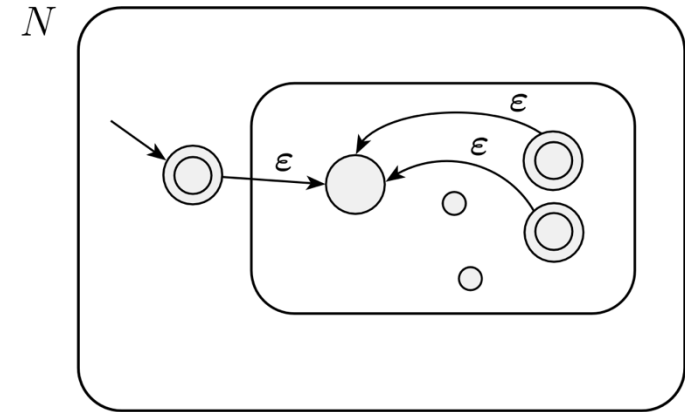
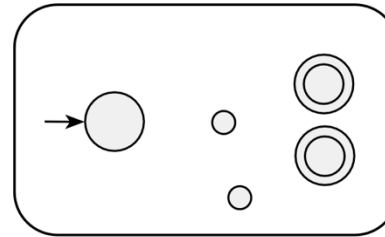
2. The state  $q_0$  is the new start state.

3.  $F = \{q_0\} \cup F_1$ .

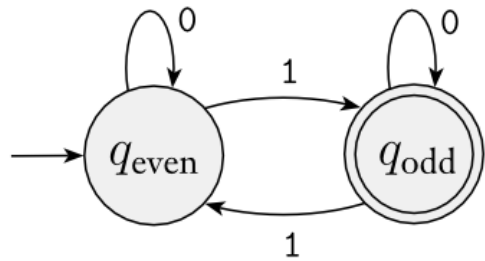
The accept states are the old accept states plus the new start state.

4. Define  $\delta$  so that for any  $q \in Q$  and any  $a \in \Sigma_\epsilon$ ,

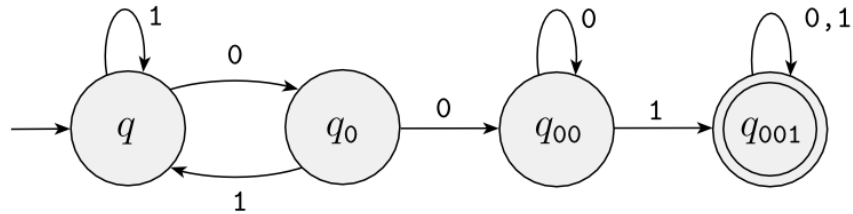
$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ and } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \epsilon \\ \{q_1\} & q = q_0 \text{ and } a = \epsilon \\ \emptyset & q = q_0 \text{ and } a \neq \epsilon. \end{cases}$$



# Closure Properties: Star Example



Language  $D = \{w \mid w \text{ has an odd number of 1s}\}$

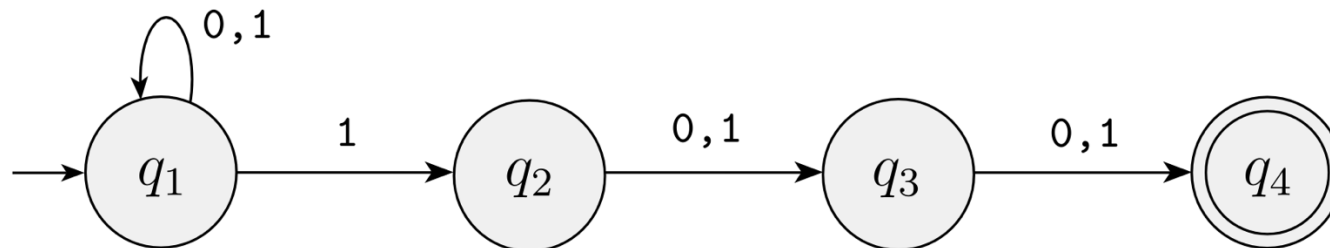


Language  $E = \{w \mid w \text{ contains } 001 \text{ as a substring}\}$

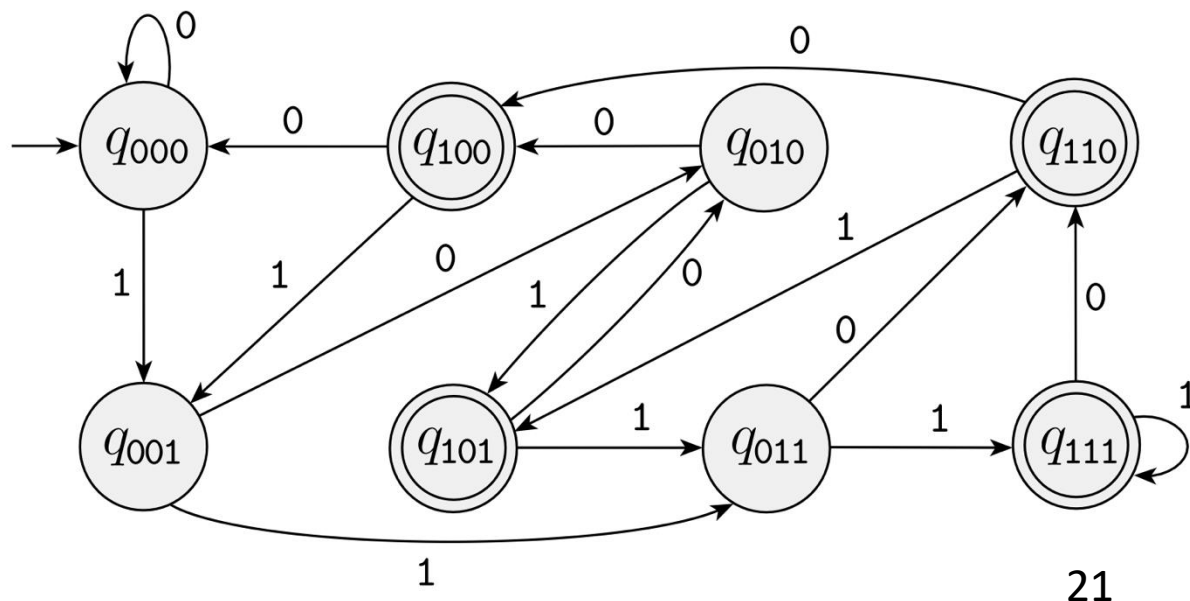
Language  $(DE)^* = \{w^* \mid w = xy \text{ where } x \text{ has an odd number of 1s and } y \text{ contains } 001 \text{ as a substring}\}$

# NFAs: Last Example

- The NFA  $N_3$  that recognises the language over  $\{0,1\}$  containing a 1 in the third position from the end



- It stays in the start state until it "guesses" that it is three places from the end ("magical")
- This is the smallest equivalent DFA, containing 8 states:



- Designing the NFA was much easier
- Understanding the behaviour of the NFA is much easier
- Non-determinism is a very useful tool to understand the properties of computation, yet not applicable in practical applications.

# Quick review of today

1. Nondeterministic finite automata (NFA)
2. Proved: NFA and DFA are equivalent in power
3. Proved: Class of regular languages is closed under  $\circ, *$